

Question 16

Q: A high-resolution image consumes excessive storage and processing time. Explain how sampling and quantization can be adjusted to optimize performance without significant quality loss.

Theoretical Concept: Optimizing Storage & Quality

To reduce file size without losing perceived quality, we adjust **sampling** and **quantization** smartly based on human vision:

- 1. Chroma Subsampling:** The human eye is less sensitive to color than brightness. We downsample color channels while keeping the brightness channel high-res (used in JPEG compression).
- 2. Vector Quantization:** Grouping similar colors together to reduce the total number of bits needed per pixel.

OpenCV Python Solution

[Copy](#)

```
import cv2 # Import OpenCV for image processing algorithms
import numpy as np # Import Numpy for mathematical mat
import matplotlib.pyplot as plt # Import Matplotlib to visualize th

# Step 1: Create a high frequency test pattern (sharp diagonal lines)
img = np.zeros((100, 100), dtype=np.uint8) # Create a blank black image array filled with zeros
for i in range(-100, 100, 10):
    cv2.line(img, (i, 0), (i+100, 100), 255, 2) # Draw a straight line on the image

# Step 2: Compress the image aggressively in-memory (JPEG Quality = 5/100)
# This forces massive quantization and chroma subsampling mathematically
encode_param = [int(cv2.IMWRITE_JPEG_QUALITY), 5]
result, encimg = cv2.imencode('.jpg', img, encode_param) # Encode image buffer into compressed

# Step 3: Decode back to pixels to view the compression artifacts
decimg = cv2.imdecode(encimg, 0) # Decode a compressed buffer back into image array

# Step 4: Show the visual loss caused by optimizing storage
plt.figure(figsize=(8,4)) # Create a new figure window for display
plt.subplot(121), plt.imshow(img, cmap='gray'), plt.title('Original (High Data)') # Place this
```

```
plt.subplot(122), plt.imshow(decimg, cmap='gray'), plt.title('JPEG Artifacts (Low Data)') # Pla
plt.show() # Show the final plot window to the user
```

Expected Output

You will see crisp lines on the left, but heavily artifacted, blocky, 'ringing' patterns on the right, visibly proving the destructive nature of aggressive lossy compression.

How to Execute & Submit

1. Google Colab Execution:

- Open [Google Colab](#) and create a New Notebook.
- Click the  **Copy** button on the code block above.
- Paste it into a Colab cell and press **Shift + Enter** to run.

2. Uploading to GitHub:

- In Colab, go to **File > Save a copy in GitHub**.
- Authorize your GitHub account.
- Select your Computer Vision Lab repository and click **OK** to commit the code.